

TECHNICAL DOCUMENTATION

Jummikville Academy

Fee Management System

A comprehensive guide to the frontend, backend, database, integrations, security, and deployment of the system built for Jummikville Academy, Uyo.

System	Jummikville Academy Fee Management System
Version	1.0.0
Built By	CAR Hub
Backend	Python 3.11+ / FastAPI / SQLAlchemy
Database	SQLite (Dev) / PostgreSQL (Production)
Frontend	Vanilla HTML5 / CSS3 / JavaScript (SPA)
Date	September 2025

Table of Contents

1	Project Overview	3
2	System Architecture	4
3	Frontend -- HTML / CSS / JavaScript SPA	5
4	Backend -- FastAPI Application	7
4.1	Application Entry Point (main.py)	7
4.2	Configuration & Environment Variables	8
4.3	API Routers & Endpoints	8
4.4	Services Layer	10
4.5	Utility Helpers	11
5	Database Layer	12
5.1	Schema Overview	12
5.2	Table Descriptions	12
5.3	Entity Relationship Summary	15
6	Authentication & Security	16
7	External Integrations	17
7.1	Paystack (Online Payments)	17
7.2	Twilio (WhatsApp Messaging)	17
8	Key Business Rules	18
9	Reports & PDF Generation	19
10	Deployment & Environment Setup	20
11	Testing	21
12	Project File Structure	22
13	Glossary	23

Note: Page numbers are approximate. Refer to section headings to navigate.

1. Project Overview

The Jummikville Academy Fee Management System is a full-stack web application built to automate and streamline school fee collection, tracking, staff payroll management, and financial reporting for Jummikville Academy, Uyo. It was built by CAR Hub.

1.1 What the System Does

Tracks fees per student	Records what every student owes per term, per fee category, per school section.
Accepts online payments	Integrates with Paystack so parents can pay through a WhatsApp payment link.
Records offline payments	Staff record cash, POS, and bank-transfer payments directly in the dashboard.
Sends WhatsApp notifications	Automatic payment confirmations and friendly balance reminders via Twilio.
Manages staff & payroll	Tracks staff members, salary schedules per term, and payout transactions.
Tracks school expenses	Records money going OUT (textbooks, repairs, fuel, etc.) for net-cash reporting.
Generates PDF reports	Auto-generated PDF receipts, payslips, and financial summary reports.
Multi-section dashboard	Separate views for Preschool, Primary, and Smart Skills High School sections.

1.2 Technology Stack

Technology	Version / Detail	Purpose
Python	3.11+	Core programming language
FastAPI	>= 0.110.0	Web framework & REST API
SQLAlchemy	>= 2.0.0	Database ORM
Alembic	>= 1.13.0	Database schema migrations
SQLite	Built-in	Development database
PostgreSQL	Any	Production database option
Uvicorn	>= 0.27.0	ASGI web server
Paystack	REST API	Online payment processing (NGN)
Twilio	>= 9.0.0	WhatsApp messaging
PyJWT	>= 2.8.0	JWT session tokens
fpdf2	>= 2.7.0	PDF generation (receipts, reports)
httpx	>= 0.27.0	Async HTTP client for external APIs
pydantic-settings	>= 2.1.0	Environment variable management
HTML5 / CSS3 / JS	Vanilla	Frontend single-page application
pytest	>= 8.0.0	Automated testing framework

1.3 School Sections

The school is divided into three sections, each with its own classes:

Preschool

Preschool 1, Preschool 2, Preschool 3, Reception

Primary

Primary 1, Primary 2, Primary 3, Primary 4, Primary 5, Primary 6

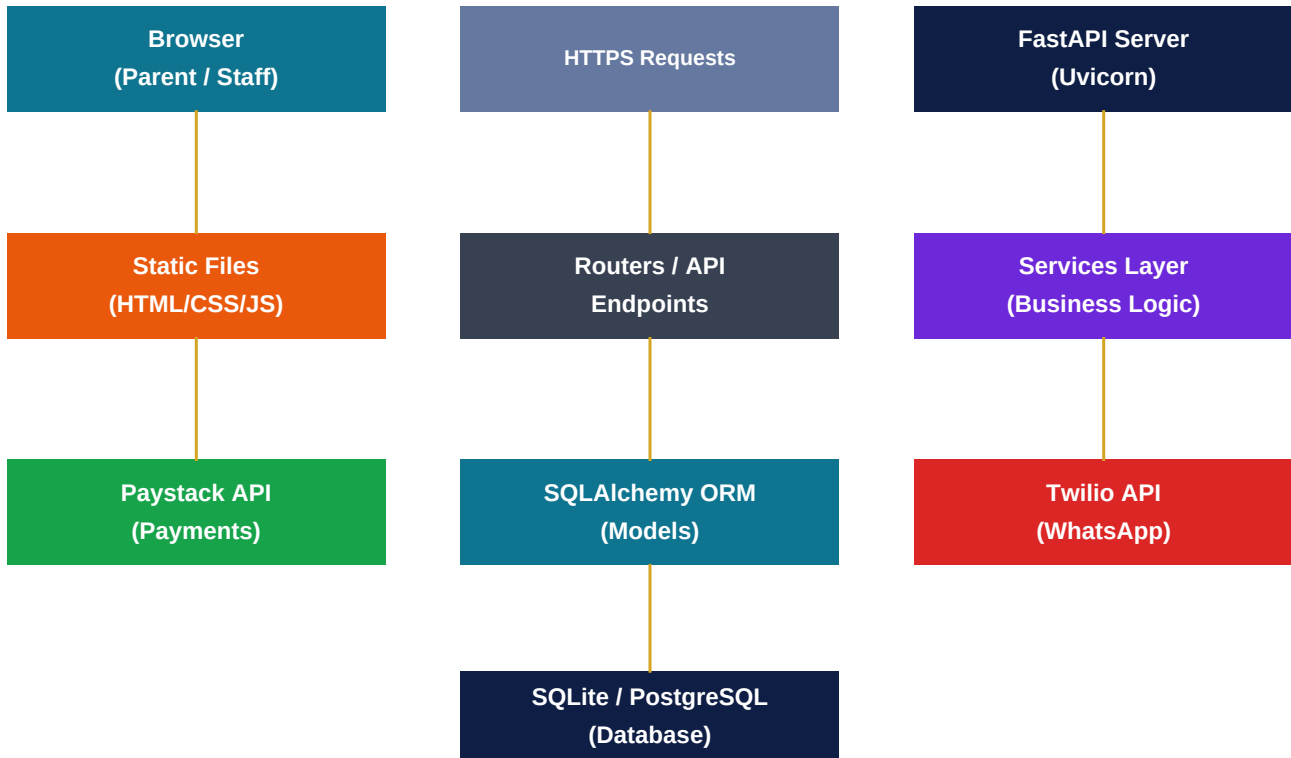
Smart Skills High School

JSS1, JSS2, JSS3, SS1, SS2, SS3

2. System Architecture

The application follows a classic monolithic MVC-style architecture where the backend serves both the REST API and the frontend static files from a single Python process. This simplifies deployment -- one server, one port, no separate CDN needed.

2.1 High-Level Architecture



2.2 Request Lifecycle

- 1. Browser sends HTTP request to `http://localhost:8000/`
- 2. Uvicorn (ASGI server) receives and routes it to FastAPI.
- 3. If the path starts with `/api/v1/`, FastAPI matches a Router endpoint.
- 4. The auth dependency verifies the session JWT cookie.
- 5. The router calls the appropriate Service function (business logic).
- 6. The Service interacts with the Database via SQLAlchemy Session.
- 7. Response is serialised via Pydantic schemas and returned as JSON.
- 8. For non-API paths, FastAPI serves the static frontend files.

2.3 Multi-Tenancy Design

Every record in the database carries a `school_id` foreign key. Although currently only one school (Jummikville Academy) is running, the architecture is fully multi-tenant. Adding a second school is a single INSERT into the schools table -- no code changes needed. Each API request filters by `school_id` to ensure data isolation.

3. Frontend -- HTML / CSS / JavaScript SPA

The frontend is a Single-Page Application (SPA) built with pure/vanilla HTML5, CSS3, and JavaScript. It is NOT a Node.js project -- there is no package.json, no npm, no build step, and no framework like React or Vue. The FastAPI server serves it directly as static files.

3.1 File Structure

File / Folder	Size	Purpose
frontend/index.html	~176 KB	The entire UI -- all pages, modals, and HTML templates
frontend/app.js	~66 KB	All JavaScript logic, API calls, and routing
frontend/assets/	--	Static assets (images, icons, fonts)

3.2 SPA Routing & Pages

Navigation is controlled entirely by JavaScript. Clicking a menu item shows the relevant section (div) and hides all others -- no page reload needed. The pages are:

Dashboard	Summary cards (collected, outstanding, expenses, net), section/class filter tabs, student overview table.
Students	Add/edit/deactivate students. Filter by section, class, and payment status. Shows fee summary per student.
Fees Setup	Create fee types per section/term, bulk-assign fees to all students in a section, manage fee categories.
Payments	Record cash/POS/bank-transfer payments. Initialize Paystack online payment links. View payment history.
Expenses	Log school expenditure by category (Textbooks, Repairs, Fuel, etc.). View and filter expense history.
Payroll	Manage staff salary schedules per term. Record payout transactions. Download PDF payslips.
Staff	Add/edit/deactivate staff members. Store bank details. View assigned classes.
Reports	Generate and download PDF financial reports. Term-based income/expense summaries.
Terms	Create academic terms. Set the current active term. View term history.
Activity Log	Audit log of all system actions (who did what and when).
Users / Settings	Add dashboard users (bursar, staff). Manage school profile and fee categories.

3.3 API Communication

All API calls are made from app.js using the browser's native fetch() API. Since the frontend and backend are served from the same origin (same server, same port), there are no CORS issues in production. The session JWT is stored as a cookie, so credentials are automatically sent with every request.

3.4 Authentication Flow (Frontend)

- 1. User enters email and password on the login page.
- 2. app.js POSTs to /api/v1/auth/login with JSON credentials.
-

3. On success, the server sets an httpOnly cookie with the JWT token.
- 4. app.js checks /api/v1/auth/me to verify session on every page load.
- 5. If the cookie is expired or missing, the user is redirected to login.
- 6. Logout calls /api/v1/auth/logout which clears the cookie server-side.

3.5 Key JavaScript Functions

app.js is organized into modules by feature area:

loadDashboard()	Fetches and renders summary cards, section tabs, and student table.
loadStudents()	Lists students with filtering and pagination.
recordPayment()	Submits cash/POS payment and refreshes the student record.
initializePaystack()	Calls the backend to generate a Paystack payment URL and shows it.
loadPayroll()	Loads staff payroll schedules for the current term.
generateReport()	Triggers PDF report generation and downloads it.
sendReminders()	Triggers batch WhatsApp reminder messages for outstanding balances.
showPage(name)	SPA routing -- shows the named page and hides all others.

4. Backend -- FastAPI Application

The backend is a Python FastAPI application organised into logical layers: Routers (endpoints), Services (business logic), Models (database tables), Schemas (data validation), and Utils (helpers). This separation of concerns keeps each layer focused and testable.

4.1 Application Entry Point (app/main.py)

- Creates the FastAPI() app instance with title, version, and docs URLs.
- Configures CORS middleware (permissive in dev, restricted in production).
- Registers all API routers -- public routes (auth, webhooks) and protected routes.
- On startup: creates all database tables (Base.metadata.create_all).
- On startup: seeds the default admin user if one doesn't exist.
- On startup: seeds 8 default fee categories (Tuition, Textbook, Lesson, etc.).
- Mounts the frontend/ directory as static files at the root URL '/'.
- Exposes /api (info), /health (monitoring), /docs (Swagger UI), /redoc.

4.2 Configuration (app/config.py)

All configuration is loaded from environment variables / a .env file using Pydantic's BaseSettings. This means no secrets in source code.

Setting	Default	Description
DATABASE_URL	sqlite:///./jummikville.db	Database connection string
PAYSTACK_SECRET_KEY	sk_test_placeholder	Paystack secret API key
PAYSTACK_PUBLIC_KEY	pk_test_placeholder	Paystack public API key
TWILIO_ACCOUNT_SID	AC_placeholder	Twilio account identifier
TWILIO_AUTH_TOKEN	placeholder	Twilio authentication token
TWILIO_WHATSAPP_NUMBER	+14155238886	WhatsApp sender number
JWT_SECRET	(dev default)	Secret key for JWT signing
SESSION_TIMEOUT_MINUTES	30	Idle session timeout
ADMIN_EMAIL	admin@jummikville.sch	Seed admin email
ADMIN_PASSWORD	(random)	Seed admin password
APP_ENV	development	'development' or 'production'

4.3 API Routers & Endpoints

Each router file in app/routers/ handles a specific domain. All protected routes require a valid JWT cookie (enforced by the get_current_user dependency injected at the router level in main.py).

auth.py [/api/v1/auth]

Method	Path	Description
POST	/login	Authenticate user and set session cookie
POST	/logout	Clear session cookie
GET	/me	Return current user info

students.py [/api/v1/students]

Method	Path	Description
POST	/	Create a new student
GET	/	List students (filter by section/class/status)
GET	/[{id}]	Get a specific student
PATCH	/[{id}]	Update student details

fees.py [/api/v1/fees]

Method	Path	Description
POST	/types	Create a fee type
GET	/types	List fee types
POST	/records	Assign a fee to one student
POST	/records/bulk	Assign a fee to all students in a section
GET	/records	List fee records
GET	/records/{id}	Get a specific fee record
GET	/categories	List fee categories
POST	/categories	Create a fee category

payments.py [/api/v1/payments]

Method	Path	Description
POST	/cash	Record cash/POS/transfer payment
POST	/initialize	Generate Paystack payment link
GET	/[{id}]/receipt	Download PDF receipt
GET	/mock-checkout-page/{ref}	Dev mock checkout page

webhooks.py [/api/v1/webhooks]

Method	Path	Description
POST	/paystack	Receive Paystack payment confirmation

dashboard.py [/api/v1/dashboard]

Method	Path	Description
GET	/summary	Summary cards + chart data
GET	/students	Student list with fee status
GET	/students/{id}	Full student detail + history
GET	/insights	Financial insights for current term
GET	/sections	Per-section breakdown
GET	/categories	Per-fee-category breakdown

payroll.py [/api/v1/payroll]

Method	Path	Description
GET	/	List payroll schedules
POST	/	Create payroll schedule
GET	/summary	Payroll summary for a term
POST	/[{id}]/pay	Record a payout transaction
DELETE	/[{id}]	Delete a payroll schedule
GET	/[{id}]/payslip	Download PDF payslip

staff.py [/api/v1/staff]

Method	Path	Description
POST	/	Create a staff member
GET	/	List staff members
GET	/[{id}]	Get a staff member
PATCH	/[{id}]	Update a staff member

expenses.py [/api/v1/expenses]

Method	Path	Description
POST	/	Record an expense
GET	/	List expenses
GET	/categories	List expense categories
GET	/summary	Expense summary by category

terms.py [/api/v1/terms]

Method	Path	Description
POST	/	Create a term
GET	/	List terms
POST	/[{id}]/set-current	Set a term as the current one

reports.py [/api/v1/reports]

Method	Path	Description
GET	/pdf	Download full financial PDF report
GET	/summary	JSON report summary

reminders.py [/api/v1/reminders]

Method	Path	Description
POST	/send	Send WhatsApp reminders to parents with balances

users.py [/api/v1/users]

Method	Path	Description
POST	/	Create a dashboard user
GET	/	List users
PATCH	/[{id}]	Update a user
DELETE	/[{id}]	Deactivate a user

activity.py [/api/v1/activity]

Method	Path	Description
GET	/	List activity log entries

4.4 Services Layer (app/services/)

Services contain the business logic, keeping routers thin. Each service module handles a specific concern:

auth_deps.py	FastAPI dependency that reads and validates the JWT cookie on every protected request. Re-issues a fresh token to reset the idle timeout.
security.py	Password hashing (PBKDF2-SHA256, 200k iterations, random salt). JWT

	creation and verification (HS256).
paystack.py	Webhook signature verification (HMAC-SHA512). Transaction initialization and verification via Paystack REST API. Mock checkout page for dev mode.
twilio_wa.py	Sends WhatsApp messages (payment confirmations, balance reminders, salary notifications) via Twilio's REST API.
payment_service.py	Records payments (cash and Paystack), updates fee record status, triggers WhatsApp confirmation, generates PDF receipt.
receipt_service.py	Generates PDF payment receipts using fpdf2. Includes school header, student details, payment breakdown, and QR placeholder.
report_service.py	Computes financial summaries (income by category, expenses, net cash, section breakdowns) for the reporting page.
report_pdf.py	Generates the full financial PDF report (Part A-E: overview, income by category, expenses, payroll, section breakdown).
payroll_pdf.py	Generates a PDF payslip for an individual staff salary payment.
payroll_report_pdf.py	Generates a batch payroll summary PDF for an entire term.
term_service.py	Handles setting the current term (atomic -- ensures exactly one term is current per school).
expense_service.py	Calculates total expenses (by term, by category) using SQL aggregation.
reminder_service.py	Fetches students with outstanding balances and sends WhatsApp reminders via Twilio.
activity_service.py	Logs system actions (who, what, when) to the activity_logs table.

4.5 Utility Helpers (app/utills/)

The utils package provides small, reusable formatting helpers:

kobo_to_naira(kobo)	Converts an integer kobo amount to a formatted Naira string. E.g., 7500000 -> 'NGN75,000.00'
whatsapp_number(phone)	Normalises a Nigerian phone number to E.164 WhatsApp format. E.g., '0802...' becomes 'whatsapp:+2348...'

5. Database Layer

The database is accessed through SQLAlchemy 2.0 ORM using Python-style mapped classes. In development, SQLite is used (a single .db file -- no setup required). In production, switch to PostgreSQL by changing DATABASE_URL in .env. The ORM layer abstracts the difference -- the same Python code works for both.

5.1 Schema Overview

The database contains the following tables:

Table	Model Class	Description
schools	School	The school entity. Root of the multi-tenant hierarchy.
users	User	Dashboard users (admin/bursar). Login accounts with hashed passwords.
students	Student	Student records with parent contact info (phone, email).
fee_categories	FeeCategory	Named categories: Tuition, Textbook, Lesson, etc.
terms	Term	Academic terms. One per school is marked is_current.
fee_types	FeeType	A fee amount for a specific category + section + term.
fee_records	FeeRecord	One student's obligation to pay one FeeType. Tracks status.
payments	Payment	Every payment event (cash, POS, Paystack). Linked to a FeeRecord.
expenses	Expense	School expenditure records (money going OUT).
payroll	Payroll	Legacy: simple staff salary payment records (Director-only).
staff	Staff	Staff member directory with bank details and assigned classes.
staff_payroll_records	StaffPayrollRecord	Term salary schedule per staff member.
staff_payments	StaffPayment	Individual payout transactions against a salary schedule.
webhook_events	WebhookEvent	Idempotency log for Paystack webhook events.
activity_logs	ActivityLog	Audit trail: who did what and when.

5.2 Table Descriptions

schools

The root entity. Every other table links back to a school_id.

Column	Type	Notes
id	INTEGER PK	Auto-increment primary key
name	VARCHAR(200)	e.g. 'Jummikville Academy'
slug	VARCHAR(50) UNIQUE	URL-safe identifier e.g. 'jummikville'
phone	VARCHAR(120)	Contact numbers (comma-separated)
email	VARCHAR(200)	School contact email
address	VARCHAR(500)	Physical address
is_active	BOOLEAN	Soft-disable a school without deleting
created_at	DATETIME	UTC timestamp

students

Column	Type	Notes
id	INTEGER PK	Auto-increment primary key

school_id	INTEGER FK	-> schools.id
student_name	VARCHAR(200)	Full name of the student
section	VARCHAR(50)	'Preschool', 'Primary', or 'Smart Skills High School'
class_name	VARCHAR(50)	e.g. 'JSS 2', 'Primary 4'
parent_name	VARCHAR(200)	Parent/guardian full name
parent_phone	VARCHAR(20)	E.164 format: +234XXXXXXXXXX
parent_email	VARCHAR(200)	Optional. Needed for Paystack transactions
is_active	BOOLEAN	Soft delete (inactive students kept for history)
created_at	DATETIME	UTC timestamp

fee_types (Fee Catalog)

Defines a fee amount for a specific combination of school + section + term + category. The balance rule: $\text{balance} = \text{total_fees} - \text{amount_paid}$ (always computed, never stored).

Column	Type	Notes
id	INTEGER PK	Auto-increment primary key
school_id	INTEGER FK	-> schools.id
category_id	INTEGER FK	-> fee_categories.id (e.g. Tuition)
section	VARCHAR(50)	Which school section this applies to
term_id	INTEGER FK	-> terms.id (source of truth for term)
amount_kobo	INTEGER	Fee amount in kobo. NGN75,000 = 7,500,000
created_at	DATETIME	UTC timestamp

fee_records (Per-Student Fee Tracking)

Column	Type	Notes
id	INTEGER PK	Auto-increment primary key
student_id	INTEGER FK	-> students.id
fee_type_id	INTEGER FK	-> fee_types.id
total_fees_kobo	INTEGER	What this student owes for this fee type
status	VARCHAR(20)	'unpaid' 'partial' 'paid' 'overpaid'
created_at	DATETIME	UTC timestamp
updated_at	DATETIME	Updated on each payment
amount_paid_kobo	COMPUTED	Sum of linked Payment.amount_kobo (NOT stored)
balance_kobo	COMPUTED	$\text{total_fees_kobo} - \text{amount_paid_kobo}$ (NOT stored)

payments

Column	Type	Notes
id	INTEGER PK	Auto-increment primary key
fee_record_id	INTEGER FK	-> fee_records.id
amount_kobo	INTEGER	Amount of THIS payment in kobo
method	VARCHAR(20)	'paystack' 'cash' 'pos' 'bank_transfer' 'other'
paystack_reference	VARCHAR(200) UNIQUE	Paystack ref -- used for idempotency
recorded_by	VARCHAR(200)	Human-readable label of who recorded it
recorded_by_user_id	INTEGER FK	-> users.id (audit trail)
receipt_url	VARCHAR(300)	URL to the generated PDF receipt
note	TEXT	Optional memo (e.g. 'Paid at school office')

paid_at	DATETIME	When payment actually occurred
created_at	DATETIME	When the record was created in the system

staff & staff_payroll_records & staff_payments

Staff stores the staff directory. StaffPayrollRecord stores the scheduled salary for a staff member for a specific term. StaffPayment stores each actual payout transaction. The same pattern as fee_records/payments is used: amount_paid_kobo is computed, not stored.

5.3 Entity Relationship Summary

```

School 1 -----< inf Students
School 1 -----< inf FeeTypes
School 1 -----< inf FeeCategories
School 1 -----< inf Terms
School 1 -----< inf Users
School 1 -----< inf Staff
Student 1 -----< inf FeeRecords
FeeType 1 -----< inf FeeRecords
FeeCategory 1 --< inf FeeTypes
Term 1 -----< inf FeeTypes
FeeRecord 1 ----< inf Payments
Staff 1 -----< inf StaffPayrollRecords
Term 1 -----< inf StaffPayrollRecords
StaffPayrollRecord 1 < inf StaffPayments

```

6. Authentication & Security

6.1 Authentication Flow

- 1. User submits email + password to POST /api/v1/auth/login.
- 2. Server looks up the User by email, verifies the PBKDF2 password hash.
- 3. On success, a JWT is created (signed with JWT_SECRET using HS256).
- 4. The JWT is returned as an httpOnly cookie (not accessible to JavaScript -- prevents XSS token theft).
- 5. Every subsequent protected API request includes the cookie automatically.
- 6. The get_current_user dependency reads the cookie, verifies the JWT, and fetches the user.
- 7. On each valid request, a fresh JWT is issued (sliding window idle timeout).
- 8. Logout clears the cookie by setting it to an empty value with immediate expiry.

6.2 Password Hashing

Passwords are hashed using PBKDF2-SHA256 (Python's standard library hashlib -- no external dependency). The hash format is:

```
pbkdf2_sha256$<iterations>$<salt_hex>$<hash_hex>
```

Parameters: 200,000 iterations, 16-byte random salt. The salt is unique per password, making rainbow-table attacks ineffective. Comparison uses hmac.compare_digest() (constant-time) to prevent timing attacks.

6.3 Paystack Webhook Security

The webhook endpoint (/api/v1/webhooks/paystack) is publicly accessible -- Paystack's servers call it when a payment succeeds. To prevent attackers from faking payment events, every webhook is verified using HMAC-SHA512:

- Paystack signs the raw request body with your secret key using HMAC-SHA512.
- The signature is sent in the 'x-paystack-signature' HTTP header.
- The server computes its own HMAC-SHA512 of the raw body using the same secret key.
- If the two signatures match (using hmac.compare_digest for timing-safety), the webhook is genuine.
- After signature verification, the server re-calls Paystack's verify API to confirm the amount actually paid.
- Webhook events are logged in the webhook_events table; duplicate references are silently skipped (idempotency).

6.4 Other Security Measures

Input Validation:	All request bodies are validated by Pydantic schemas before touching the database -- protects against malformed data.
SQL Injection:	SQLAlchemy ORM parameterizes all queries automatically -- no raw SQL string interpolation.
CORS:	In production (APP_ENV=production), CORS is locked down (empty allow_origins). In dev, all origins are allowed for convenience.
No Secrets in Code:	All API keys, passwords, and secrets are loaded from environment variables /

.env file via pydantic-settings.

Soft Deletes:

Users, students, and staff are never deleted -- they are deactivated (is_active=False), preserving audit history.

Multi-tenant Isolation:

Every query filters by school_id. A user can only see data belonging to their school.

7. External Integrations

7.1 Paystack (Online Payments)

Paystack is Nigeria's leading payment gateway. It processes card payments, bank transfers, and USSD payments from Nigerian bank accounts. Currency is NGN (Naira).

Payment Flow

- 1. Staff clicks 'Send Payment Link' for a student's outstanding fee.
- 2. Backend calls Paystack's /transaction/initialize API with the amount (in kobo), parent's email, and a unique reference.
- 3. Paystack returns an authorization_url (a payment page link).
- 4. The staff copies this link and sends it to the parent (manually or via WhatsApp).
- 5. Parent opens the link, completes payment on Paystack's secure page.
- 6. Paystack calls the webhook URL (/api/v1/webhooks/paystack) with a charge.success event.
- 7. Backend verifies the signature, then re-verifies the transaction via Paystack API.
- 8. If valid: a Payment record is created, the FeeRecord status is updated, a WhatsApp confirmation is sent to the parent, and a PDF receipt is generated.

Reference Format

Transaction references follow the format: JMK-{fee_record_id}-{unix_timestamp}

Example: JMK-42-1706547200

The fee_record_id is encoded in the reference so the webhook can find the right record without a database lookup by reference string.

Dev / Test Mode

If PAYSTACK_SECRET_KEY contains 'placeholder', the system returns a mock checkout page URL instead of calling Paystack. This allows full testing of the payment flow without a real Paystack account.

7.2 Twilio (WhatsApp Messaging)

Twilio's WhatsApp API is used to send messages to parents and staff. Currently using Twilio's WhatsApp Sandbox (for development). The sandbox number is +1-415-523-8886.

Messages Sent

Payment Confirmation	Sent to parent after every successful payment. Includes: student name, fee type, amount paid, and remaining balance.
Fee Reminder	Sent to parents with outstanding balances (triggered manually from the dashboard). Warm, respectful tone in Nigerian English.
Salary Notification	Sent to staff when their salary is paid. Includes: amount, term, and a note from the Director.

Phone Number Normalisation

Nigerian phone numbers come in many formats (0802..., +234802..., 234802...). The whatsapp_number() utility normalises all formats to E.164 (+234XXXXXXXXXX) before passing to Twilio. Twilio requires numbers prefixed with 'whatsapp:'.

Getting to Production

To move from the Twilio sandbox to a verified WhatsApp Business number:

1. Apply for a WhatsApp Business number through Twilio.
2. Set TWILIO_WHATSAPP_NUMBER=+234XXXXXXXXXX in .env.
3. That's it -- no code changes needed.

8. Key Business Rules

> Balance is always computed

`FeeRecord.balance_kobo = total_fees_kobo - sum(payment.amount_kobo)`. The balance is NEVER stored as a column -- it is derived from the actual Payment rows. This means it can never be 'wrong'. Similarly, `amount_paid_kobo` is computed, not stored.

> Payments are always positive events

A Payment row only exists when real money has been received (cash/POS recorded by staff, or a Paystack webhook that passed signature verification). There is no 'pending' payment state.

> Exactly one current term per school

`Term.is_current` is a boolean that is True for at most one term per school at any time. Setting a new term as current atomically clears the flag on all others in the same school. The current term is set manually by an admin -- never automated by date.

> Fee categories vs fee types

`FeeCategory` is the permanent label (Tuition, Textbook). `FeeType` is a specific amount for a category x section x term. This lets 'Tuition' for Primary be NGN75,000 and for Preschool be NGN50,000 in the same term.

> Idempotent webhooks

If Paystack sends the same webhook twice (it retries on failure), the system ignores the second one. The `webhook_events` table stores every Paystack reference; if a reference has already been processed, the new event is accepted with HTTP 200 but not processed again.

> Soft deletes everywhere

Students, staff, and users are never hard-deleted. `is_active=False` marks them as inactive. Historical fee records, payments, and payroll records are preserved forever.

> Kobo as the integer currency unit

All monetary values are stored as integers in kobo (1 Naira = 100 kobo). $\text{NGN75,000} = 7,500,000 \text{ kobo}$. This avoids floating-point arithmetic errors that can cause penny discrepancies in financial software.

> Multi-section fee types

A `FeeType` must specify a section. This is required because Preschool, Primary, and High School typically have different fee amounts for the same category. Bulk fee assignment only applies to students in the specified section.

> Expense records reduce net cash (not student balances)

Expense rows track school spending (going OUT). They reduce the school's Net Available (= Total Collected - Total Expenses) on the dashboard. They NEVER affect student fee balances.

> Admin is seeded on first startup

On the first server start with a fresh database, the system automatically creates the school record and an admin user. The password is either taken from `ADMIN_PASSWORD` in `.env`, or a random secure password is generated and printed to the log (once only).

9. Reports & PDF Generation

The system uses fpdf2 (already a dependency) to generate PDF documents server-side. PDFs are streamed directly to the browser -- they are not saved as files on disk except for receipts which have a stable URL.

9.1 PDF Documents Generated

Payment Receipt

Generated by: `receipt_service.py`

Generated after every payment (cash or Paystack). Contains: school header, student details, payment method, amount, and running balance. URL: `/api/v1/payments/{id}/receipt`

Financial Report

Generated by: `report_pdf.py`

Full school financial PDF. 5 parts:

- Part A: Executive Summary (total collected, paid, outstanding, net)
- Part B: Income by Fee Category (Tuition, Textbook, etc.)
- Part C: Expenses by Category
- Part D: Payroll Summary
- Part E: Per-Section Breakdown (Preschool / Primary / HS)

Staff Payslip

Generated by: `payroll_pdf.py`

Individual payslip for one staff salary payout. Contains: staff name, role, term, amount paid, payment date, bank details. URL: `/api/v1/payroll/{id}/payslip`

Payroll Summary

Generated by: `payroll_report_pdf.py`

Term-level payroll summary for all staff. Contains: total scheduled, total paid, remaining balance per staff member.

9.2 Report Service Logic

The `report_service.py` module computes all financial figures server-side using SQLAlchemy queries and Python aggregation. Key computed values:

<code>total_collected_kobo</code>	Sum of all confirmed <code>Payment.amount_kobo</code> for the school/term.
<code>total_outstanding_kobo</code>	Sum of all <code>FeeRecord.balance_kobo</code> where <code>balance > 0</code> .
<code>total_expenses_kobo</code>	Sum of all <code>Expense.amount_kobo</code> for the school/term.
<code>net_available_kobo</code>	<code>total_collected - total_expenses</code> (cash position after running costs).
<code>by_category</code>	Income grouped by <code>FeeCategory</code> (Tuition collected, Textbook collected, etc.).
<code>by_section</code>	Income and outstanding amounts split by school section.

10. Deployment & Environment Setup

10.1 Development Setup (Local)

```
git clone <repo-url>
# Clone the repository.

cd Jummikville-Academy-Fee-management-system
# Enter the project folder.

python -m venv venv
# Create a Python virtual environment.

venv\Scripts\activate (Windows)
# Activate the virtual environment.

pip install -r requirements.txt
# Install all dependencies.

copy .env.example .env
# Copy the environment template.

# Edit .env and set ADMIN_PASSWORD, PAYSTACK keys, TWILIO keys
# Fill in secrets.

uvicorn app.main:app --reload --port 8000
# Start the development server with auto-reload.

# Open http://localhost:8000 in browser
# Access the application.
```

10.2 Important URLs (Development)

URL	Description
http://localhost:8000/	Main application UI (frontend SPA)
http://localhost:8000/docs	FastAPI Swagger UI (interactive API explorer)
http://localhost:8000/redoc	ReDoc API documentation
http://localhost:8000/api	API info / health endpoint
http://localhost:8000/health	Simple health check (returns {status: ok})
http://localhost:8000/api/v1/auth/login	Login endpoint (POST)

10.3 Production Checklist

- Set APP_ENV=production in .env.
- Use PostgreSQL: set DATABASE_URL=postgresql://user:pass@host:5432/jummikville.
- Set a strong JWT_SECRET (at least 32 random characters).
- Set ADMIN_PASSWORD to a secure password.
- Set real PAYSTACK_SECRET_KEY and PAYSTACK_PUBLIC_KEY (live keys, not test).
- Set real TWILIO credentials and a verified WhatsApp Business number.
- Configure your Paystack webhook URL to: https://yourdomain.com/api/v1/webhooks/paystack.
- Run the server behind a reverse proxy (nginx) with TLS/HTTPS.
- Use a process manager (systemd or supervisor) to keep uvicorn running.
- Set CORS origins in main.py for production (replace allow_origins=[]).

10.4 Testing Paystack Webhooks Locally

Paystack needs to reach your webhook endpoint over the internet. Use ngrok:

Terminal 1: `uvicorn app.main:app --reload --port 8000`

Terminal 2: `ngrok http 8000`

Paystack Dashboard: Set `webhook` URL `to:`
`https://xxxxx.ngrok.io/api/v1/webhooks/paystack`

Event: `Subscribe to: charge.success`

11. Testing

The project uses pytest for automated tests. Tests live in the tests/ directory.

11.1 Running Tests

pytest

pytest -v **# verbose output**

pytest tests/test_payments.py **# single file**

11.2 Test Coverage Areas

- Authentication (login, logout, session expiry, invalid credentials).
- Student CRUD (create, read, update, deactivate).
- Fee type creation and bulk assignment.
- Cash payment recording and balance recalculation.
- Paystack webhook processing (valid signature, invalid signature, duplicate event).
- WhatsApp message sending (mocked Twilio client).
- Report generation (total collected, outstanding, expenses).
- Term management (create, set current, only one current at a time).

11.3 Test Utilities

Tests use pytest fixtures to create an in-memory SQLite test database, inject test data (school, students, fee records), and mock external services (Twilio, Paystack) so tests run without internet access and don't send real messages.

12. Project File Structure

```
Jummikville-Academy-Fee-management-system/
|
+-- app/                                # Backend Python package
|   +-- __init__.py
|   +-- main.py                         # FastAPI app, startup, router registration
|   +-- config.py                       # Environment-based settings (pydantic-settings)
|   +-- database.py                     # SQLAlchemy engine, session factory, Base
|   +-- constants.py                   # School sections, classes, expense categories
|   |
|   +-- models/                         # SQLAlchemy ORM models (database tables)
|   |   +-- school.py                  # School
|   |   +-- student.py                 # Student
|   |   +-- user.py                    # User (dashboard login accounts)
|   |   +-- fee_category.py            # FeeCategory
|   |   +-- term.py                    # Term
|   |   +-- fee.py                     # FeeType + FeeRecord
|   |   +-- payment.py                 # Payment
|   |   +-- expense.py                 # Expense
|   |   +-- payroll.py                 # Payroll (legacy simple payroll)
|   |   +-- staff.py                   # Staff
|   |   +-- staff_payroll.py           # StaffPayrollRecord + StaffPayment
|   |   +-- activity.py                # ActivityLog
|   |   +-- webhook_event.py           # WebhookEvent (idempotency)
|   |
|   +-- schemas/                       # Pydantic request/response schemas
|   |   +-- auth.py, student.py, fee.py, payment.py
|   |   +-- expense.py, payroll.py, staff.py, term.py
|   |   +-- dashboard.py, report.py, user.py, activity.py
|   |
|   +-- routers/                       # FastAPI route handlers
|   |   +-- auth.py, students.py, fees.py, payments.py
|   |   +-- webhooks.py, dashboard.py, expenses.py, payroll.py
|   |   +-- staff.py, terms.py, reports.py, reminders.py
|   |   +-- users.py, activity.py
|   |
|   +-- services/                      # Business logic layer
|   |   +-- auth_deps.py               # JWT cookie dependency
|   |   +-- security.py                # Password hashing + JWT
|   |   +-- paystack.py                # Paystack API integration
|   |   +-- twilio_wa.py               # WhatsApp messaging (Twilio)
|   |   +-- payment_service.py         # Payment recording orchestration
|   |   +-- receipt_service.py         # PDF receipt generation
|   |   +-- report_service.py          # Financial report computation
|   |   +-- report_pdf.py              # Financial report PDF
|   |   +-- payroll_pdf.py             # Payslip PDF
|   |   +-- payroll_report_pdf.py      # Payroll summary PDF
|   |   +-- term_service.py            # Term management
|   |   +-- expense_service.py         # Expense aggregation
|   |   +-- reminder_service.py        # WhatsApp reminder batch
|   |   +-- activity_service.py        # Audit logging
|   |
|   +-- utils/                         # Utility helpers
|   |   +-- formatting.py              # kobo_to_naira(), whatsapp_number()
|   |
+-- frontend/                          # Static frontend (served by FastAPI)
|   +-- index.html                     # Full SPA (~176 KB -- all pages in one file)
|   +-- app.js                         # All JavaScript (~66 KB)
|   +-- assets/                        # Images, icons
|
+-- tests/                             # pytest test suite
+-- storage/                           # File storage (receipts, uploads)
```

```
+-- seed.py                # Demo data population script
+-- reset_demo_data.py     # Reset to clean demo state
+-- migrate_terms.py       # One-time migration: add terms table
+-- migrate_sections.py    # One-time migration: add sections
+-- migrate_derive_payments.py # One-time migration: derive payment amounts
+-- jummikville.db         # SQLite database (development)
+-- requirements.txt       # Python dependencies
+-- .env                   # Active environment variables (NOT in git)
+-- .env.example           # Environment variable template
```

13. Glossary

Kobo	The smallest unit of Nigerian currency. 100 kobo = NGN1 Naira. All monetary values in the database are stored in kobo to avoid floating-point errors.
FeeType	A fee definition for a specific school section, term, and fee category (e.g. 'Tuition, Primary, First Term 2025/2026, NGN75,000').
FeeRecord	One student's assigned obligation to pay one FeeType. Tracks their individual total, paid amount, and balance.
Payment	An actual confirmed payment event. Every cash, POS, bank transfer, or Paystack payment creates one Payment row.
FeeCategory	The permanent label for a type of fee (Tuition, Textbook, Lesson, etc.). Not tied to any specific term.
Term	An academic term (e.g. 'First Term 2025/2026'). Exactly one term per school is marked as the current term.
Section	One of three school groupings: Preschool, Primary, or Smart Skills High School.
Balance	$\text{balance_kobo} = \text{total_fees_kobo} - \text{amount_paid_kobo}$. Always computed, never stored.
Paystack	Nigerian payment gateway used for online fee collection. Accepts cards, bank transfers, and USSD.
Twilio	Cloud communication platform used for sending WhatsApp messages to parents and staff.
JWT	JSON Web Token -- a signed token that proves a user is logged in. Stored as an httpOnly cookie.
PBKDF2	Password-Based Key Derivation Function 2. Used to hash passwords with 200,000 iterations and a random salt.
HMAC-SHA512	Hash-based Message Authentication Code using SHA-512. Used to verify Paystack webhook signatures.
ORM	Object-Relational Mapper. SQLAlchemy maps Python classes to database tables so you write Python instead of SQL.
ASGI	Asynchronous Server Gateway Interface. The Python web server standard that Uvicorn and FastAPI use.
SPA	Single-Page Application. The frontend loads once and JavaScript handles all navigation without page reloads.
Webhook	An HTTP POST request sent by Paystack to notify us when a payment succeeds.
Idempotency	Ensuring that processing the same event twice produces the same result. Achieved via the <code>webhook_events</code> table.
Soft Delete	Marking a record as inactive (<code>is_active=False</code>) instead of deleting it. Preserves historical data.
Multi-tenant	Architecture where multiple schools can share one codebase and database, with data isolated by <code>school_id</code> .
E.164	International phone number format: <code>+ [country code] [number]</code> . Example: <code>+2348021234567</code> .
NGN	ISO 4217 currency code for the Nigerian Naira.
Payslip	A PDF document given to a staff member confirming their salary payment details.

CORS

Cross-Origin Resource Sharing -- browser security that controls which domains can call an API.

Jummikville Academy

Fee Management System

Powered by CAR Hub

This document is confidential and intended solely for the use
of Jummikville Academy staff and administrators.

© 2025 CAR Hub. All rights reserved.